

Data Storage Systems

(Sistemas de Armazenamento de Dados)

Key-Value Stores



Storage Interfaces

The most common ones

● Block Device

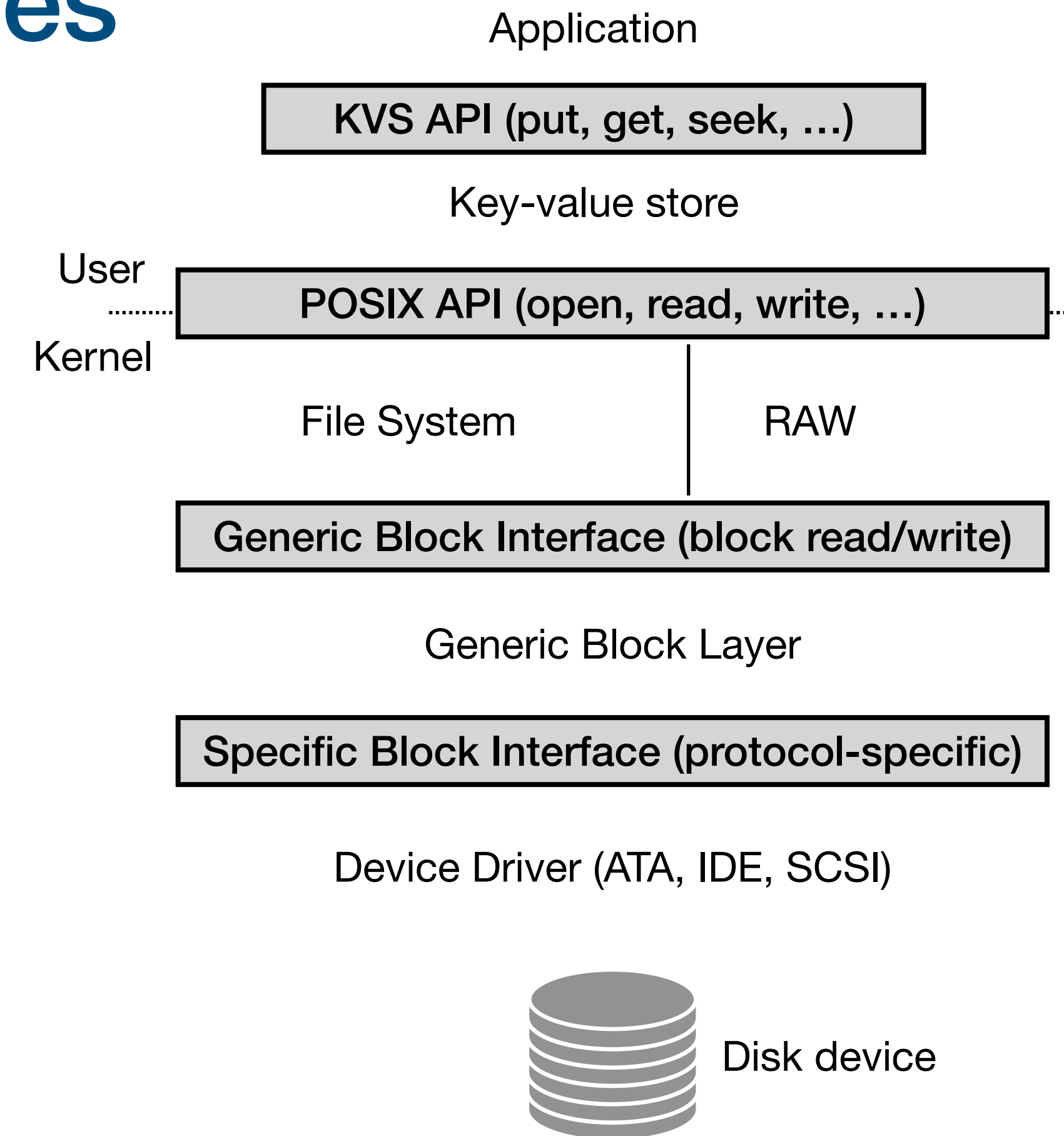
- Data is managed as a set of blocks (closest abstraction to the disk)
- Solutions: Linux block device, iSCSI

● File System (FS)

- Data is managed as a hierarchy of directories and files (the interface most users rely on their personal computers)
- Solutions: Ext4, ZFS, NFS

● Key-value Store

- Data is managed as key-value pairs (i.e., the key is a unique identifier, and the value is the content)
- Solutions: LevelDB, RocksDB



A simplified Linux File System stack

Key-value store

Overview

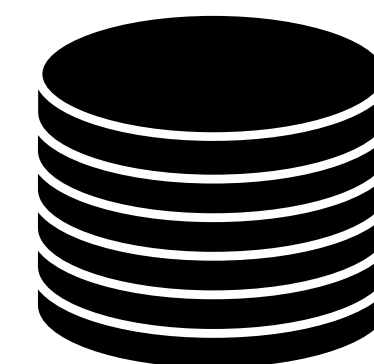
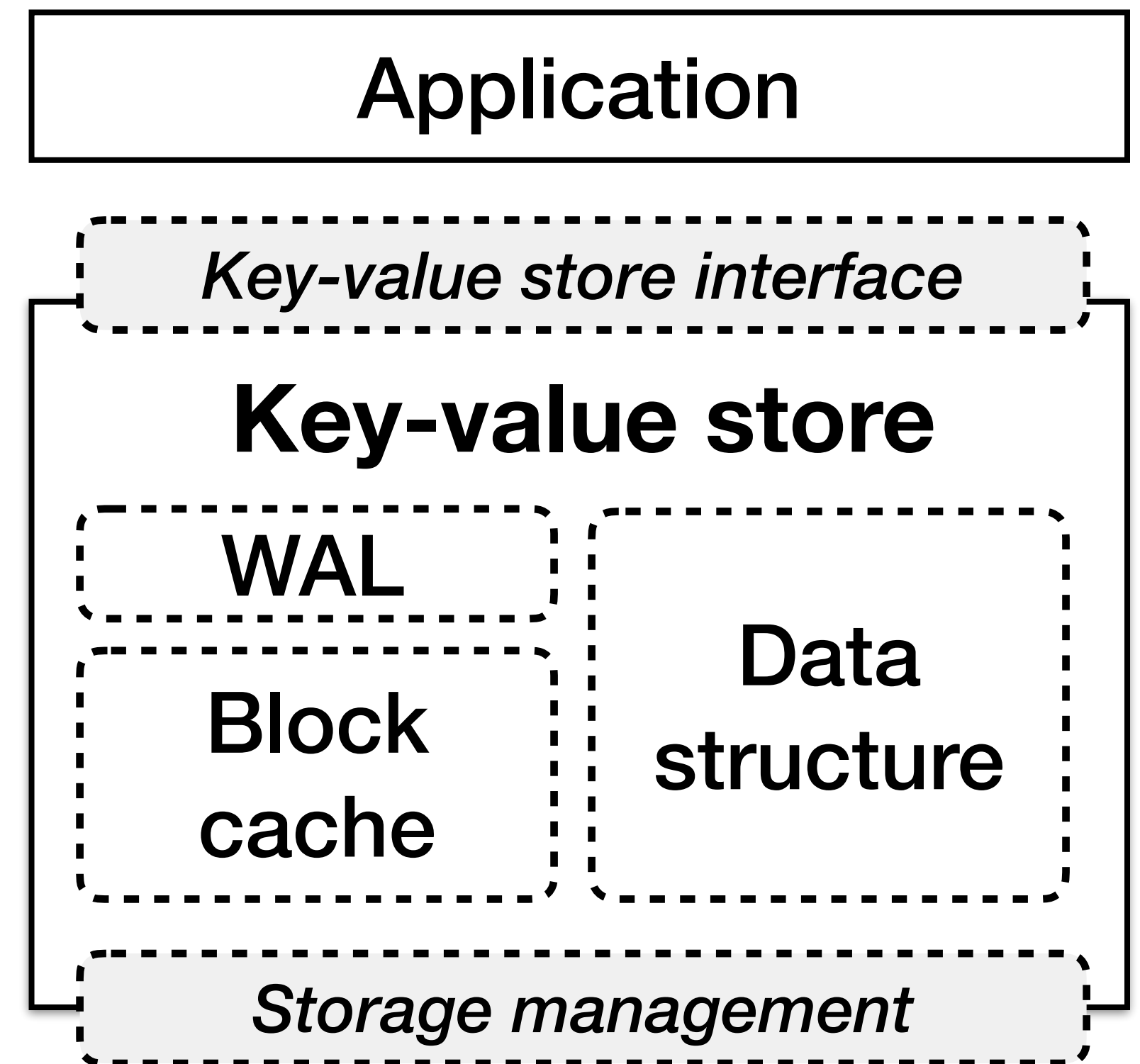
- Serve as storage engine of many data-centric systems
- Database storage engine
 - MongoDB, MyRocks, MySQL, YugaByteDB, ...
- Metadata management
 - Ceph (BlueStore), Alluxio, Pinterest, ...
- Caching and queuing
 - Netflix, Apache Spark, Uber, Milvus, ...
- And many more ...
 - Blockchains, AI engines, bidding platforms, ...

¹<https://github.com/facebook/rocksdb/wiki/RocksDB-Users-and-Use-Cases>

Key-value store

Overview

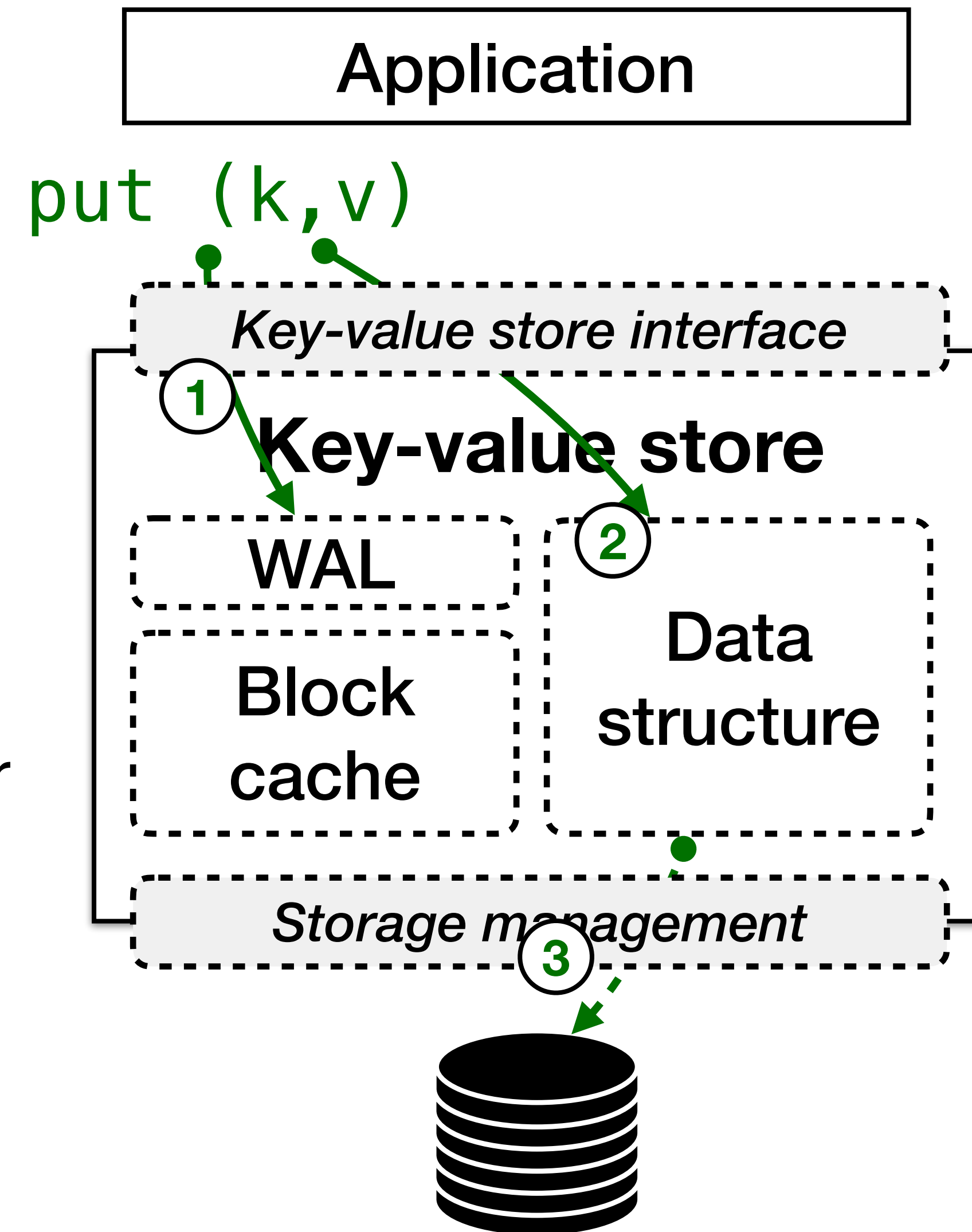
- Key-value store interface
 - Put, get, seek
- Design
 - Write-ahead log
 - Block cache
 - Data structure
 - B+tree
 - LSM
- Storage management
 - File system, block device, specialized storage



Key-value store

Interface

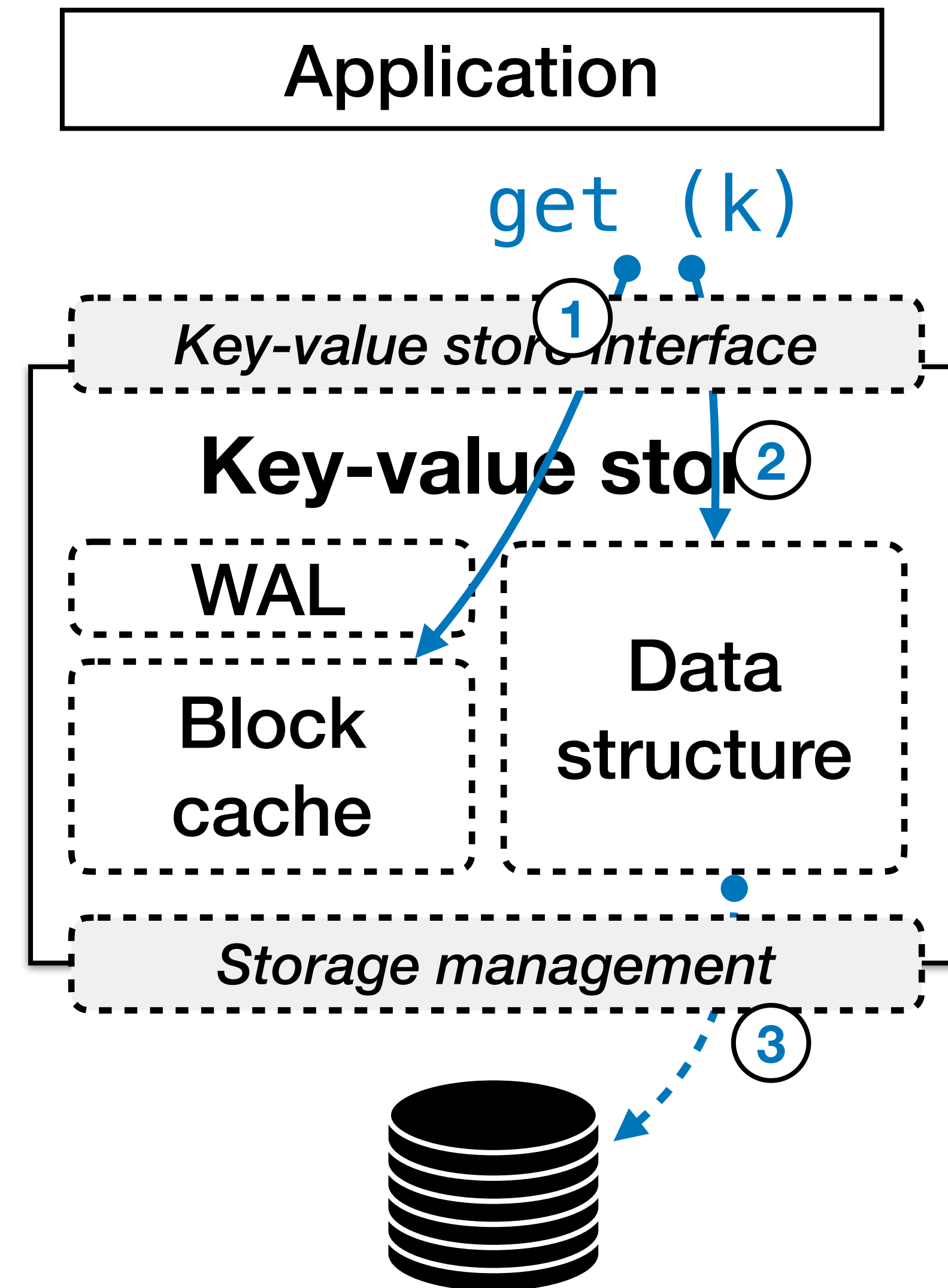
- Data is organized in a key-value pair format
 - **Keys** are often defined as integers (*uint64_t*) or strings with 8 to 16 bytes
 - **Values** are defined as strings or custom objects (e.g., images, files)
- Operations
 - **Put (key, value)**: insert a new (or update) key-value pair
 - **Get (key)**: retrieve the value of a given key
 - **Seek (start_key, end_key)**: retrieve the elements between two keys
 - **Delete (key)**: delete a key-value pair



Key-value store

Interface

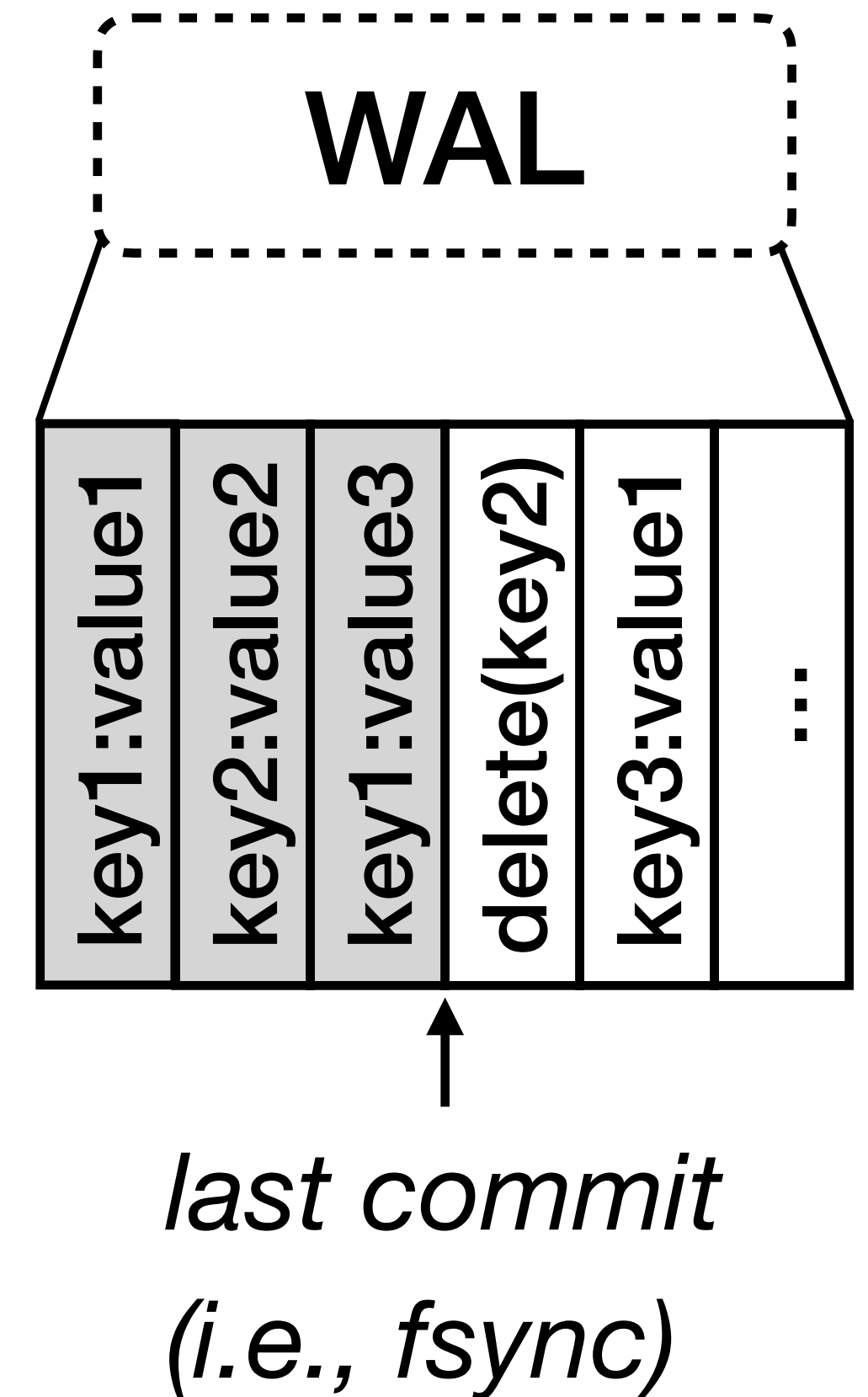
- Data is organized in a key-value pair format
 - **Keys** are often defined as integers (*uint64_t*) or strings with 8 to 16 bytes
 - **Values** are defined as strings or custom objects (e.g., images, files)
- Operations
 - **Put (key, value)**: insert a new (or update) key-value pair
 - **Get (key)**: retrieve the value of a given key
 - **Seek (start_key, end_key)**: retrieve the elements between two keys
 - **Delete (key)**: delete a key-value pair



Design

Write-ahead log (WAL)

- Also referred to as *commit log*
- Used for **crash-consistency** and **recovery**
 - *fsync after write**
- Operation mode
 - Append-only
 - Single-writer
 - Removed/recycled after reaching capacity
 - Some KVS support multiple WALs for parallelism



¹[RocksDB's Write-Ahead Log](#)

Design

Block cache

- ◎ In-memory cache for reads (*i.e.*, get, seek)
 - Typically uses an LRU caching scheme
 - Improves read latency
 - Avoids disk reads over frequently accessed data
- ◎ Stores essential data to improve read performance:
 - Uncompressed **key-value pairs**
 - **File indexes** to speed up lookups within a given file
 - **Bloom filters** to identify if a given key exists
- ◎ Varies across KVS implementations

¹[RocksDB's Block Cache](#)

Data structures

B+tree vs. LSM tree

● B+tree

- ▶ Hierarchical tree with nodes containing sorted key-value pairs
- ▶ Optimized for read-intensive workloads
- ▶ Fragmentation and poor write performance
- ▶ WiredTiger, LMDB, ...

● Log-Structured Merge (LSM) tree

- ▶ Data is organized in levels, being first written to memory and asynchronously moved down the stack
- ▶ Optimized for write-intensive workloads
- ▶ Write and read amplification
- ▶ **LevelDB**, RocksDB, Cassandra, HBase, ...

B+tree

Organization

- ◎ **B+trees** are *self-balancing, sorted tree structures*
- ◎ Contain multiple nodes, each holding up to N keys and $N+1$ pointers
 - **Root and internal nodes:** contain keys and child pointers
 - **Leaf nodes:** sit at the bottom and store the actual key-value pairs
- ◎ Provide efficient searching
 - **Put* and get:** binary search ($O(\log N)$)
 - **Seeks:** leaves are stored in a double-linked list

B+tree

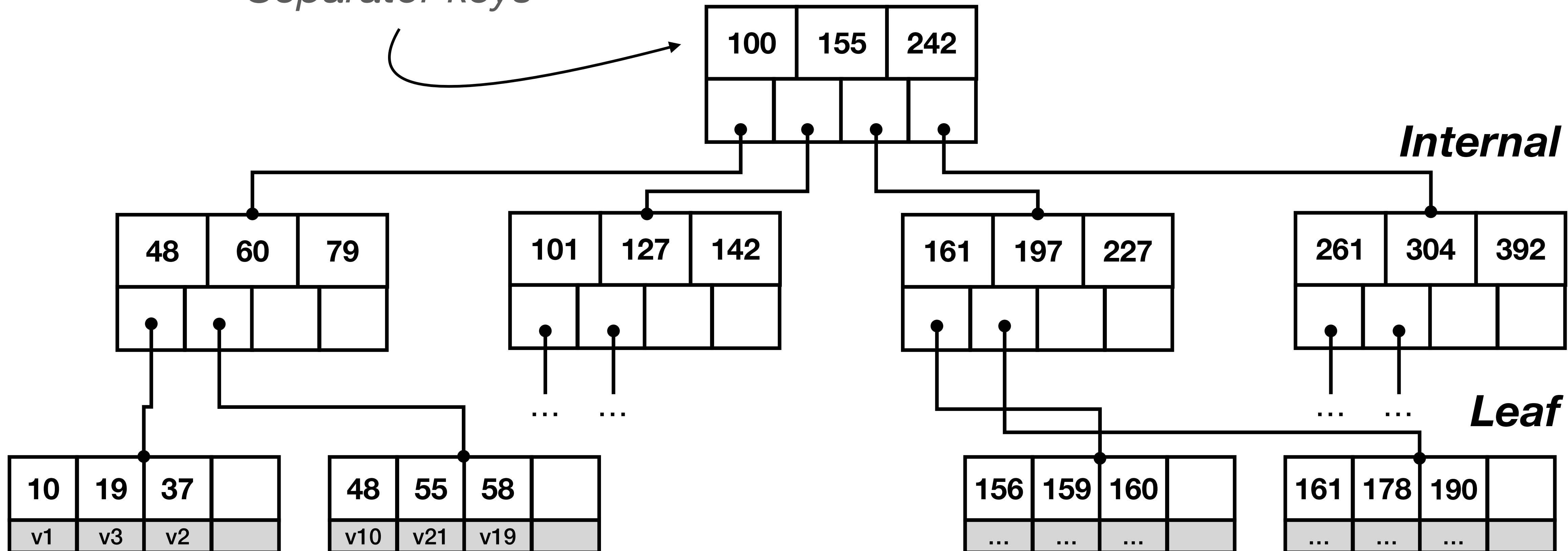
Organization

Separator keys

Root node

Internal nodes

Leaf nodes



B+tree

Get and seek operations

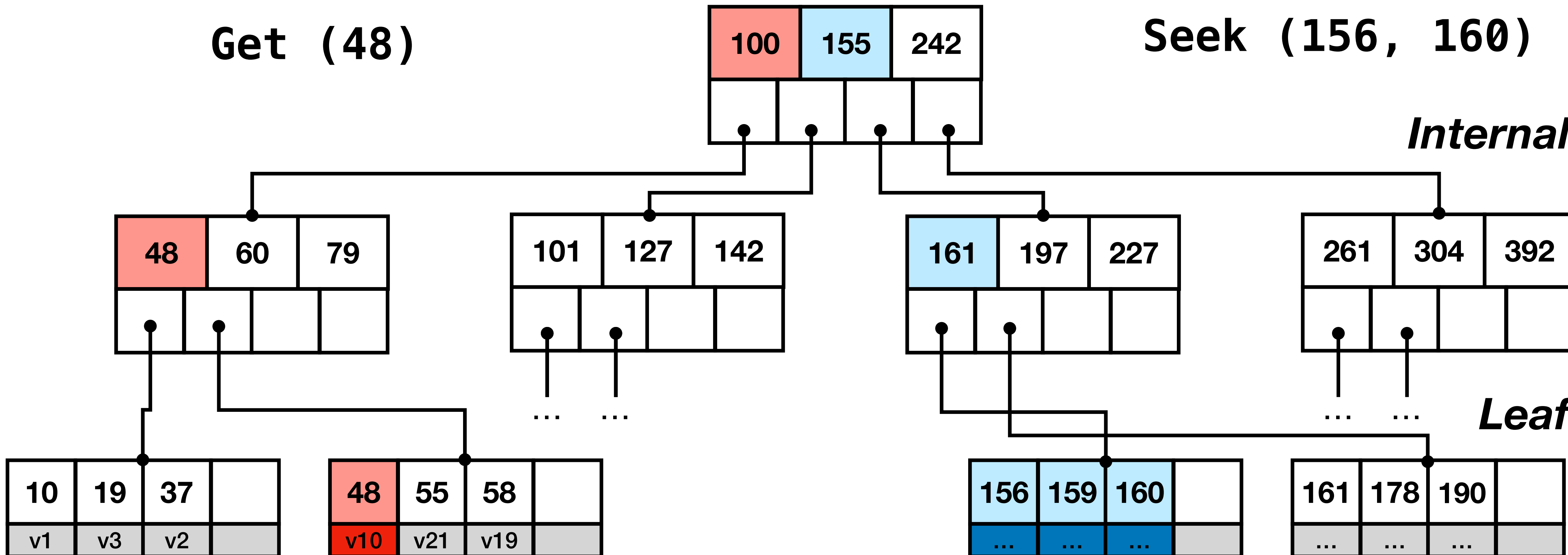
Root node

Get (48)

Seek (156, 160)

Internal nodes

Leaf nodes



B+tree

Put and delete operations

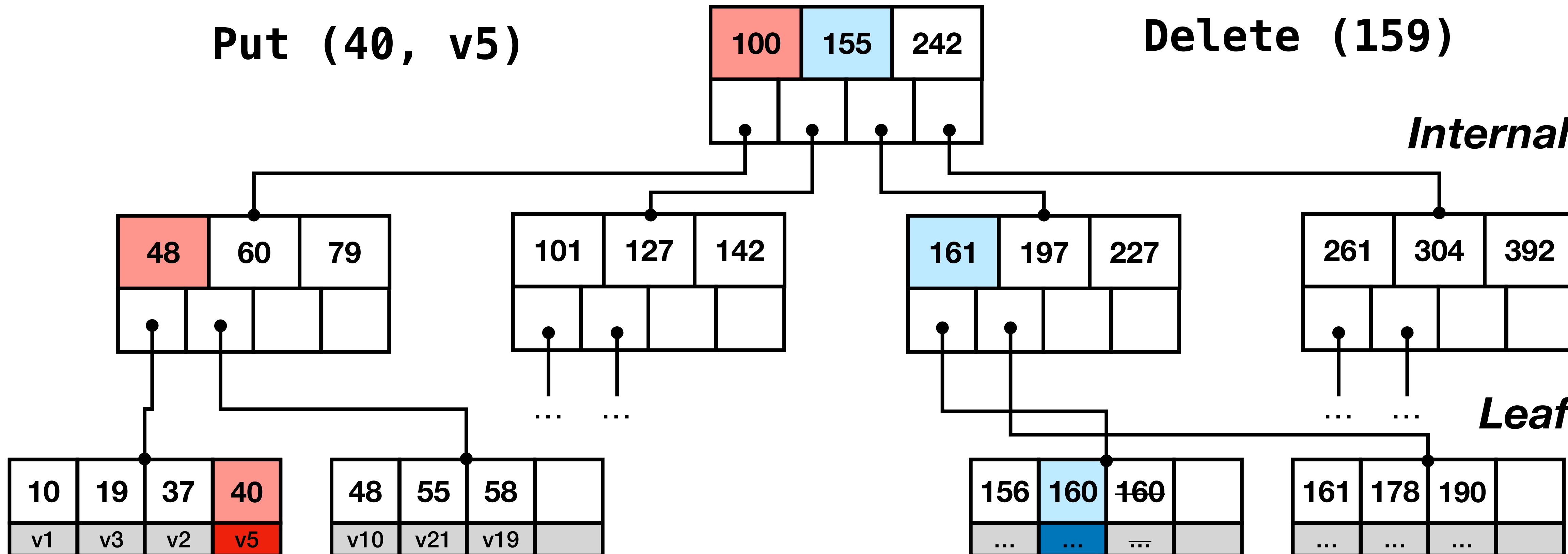
Root node

Put (40, v5)

Delete (159)

Internal nodes

Leaf nodes



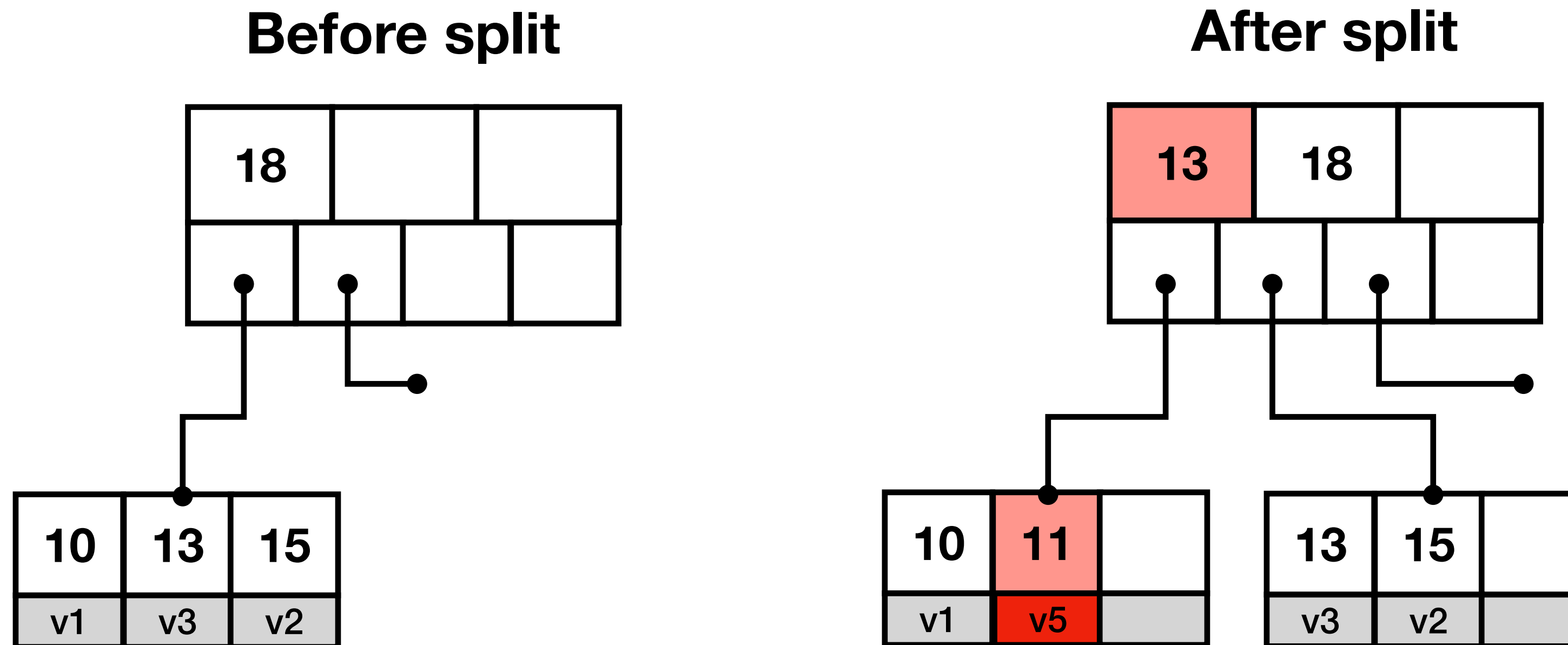
Put operations are made in-place!

B+tree

Node split

Put (11, v5)

Example of a leaf node split



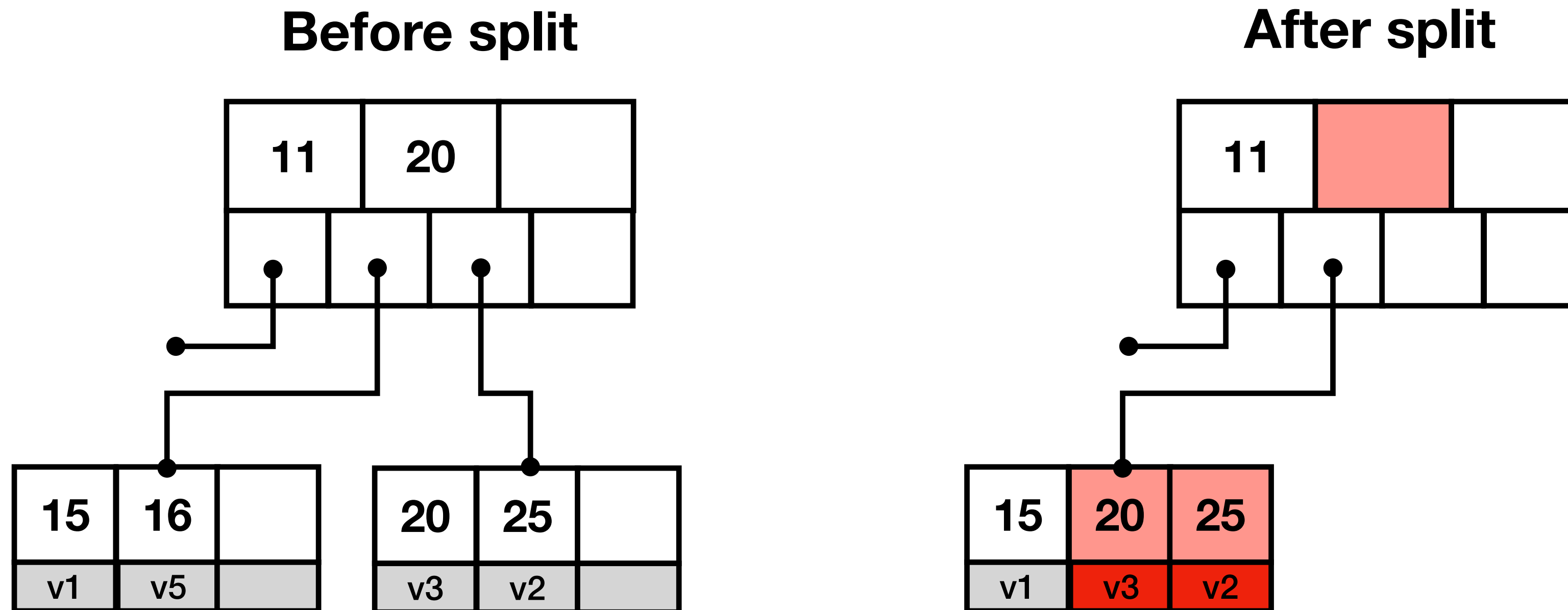
A **node split** occurs when the target node (*leaf* or *nonleaf nodes*) does **not have enough space** to hold the **new entry (overflow)**.

B+tree

Node merge

Delete (16)

Example of a leaf node merge



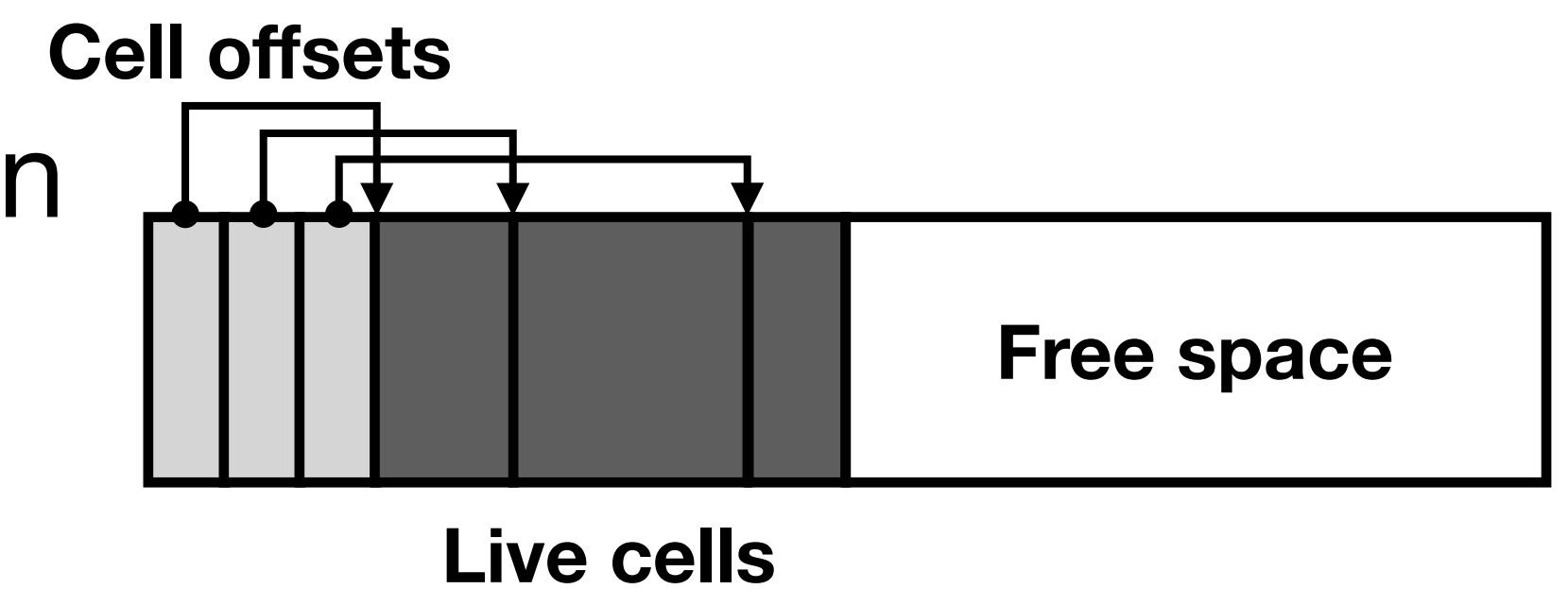
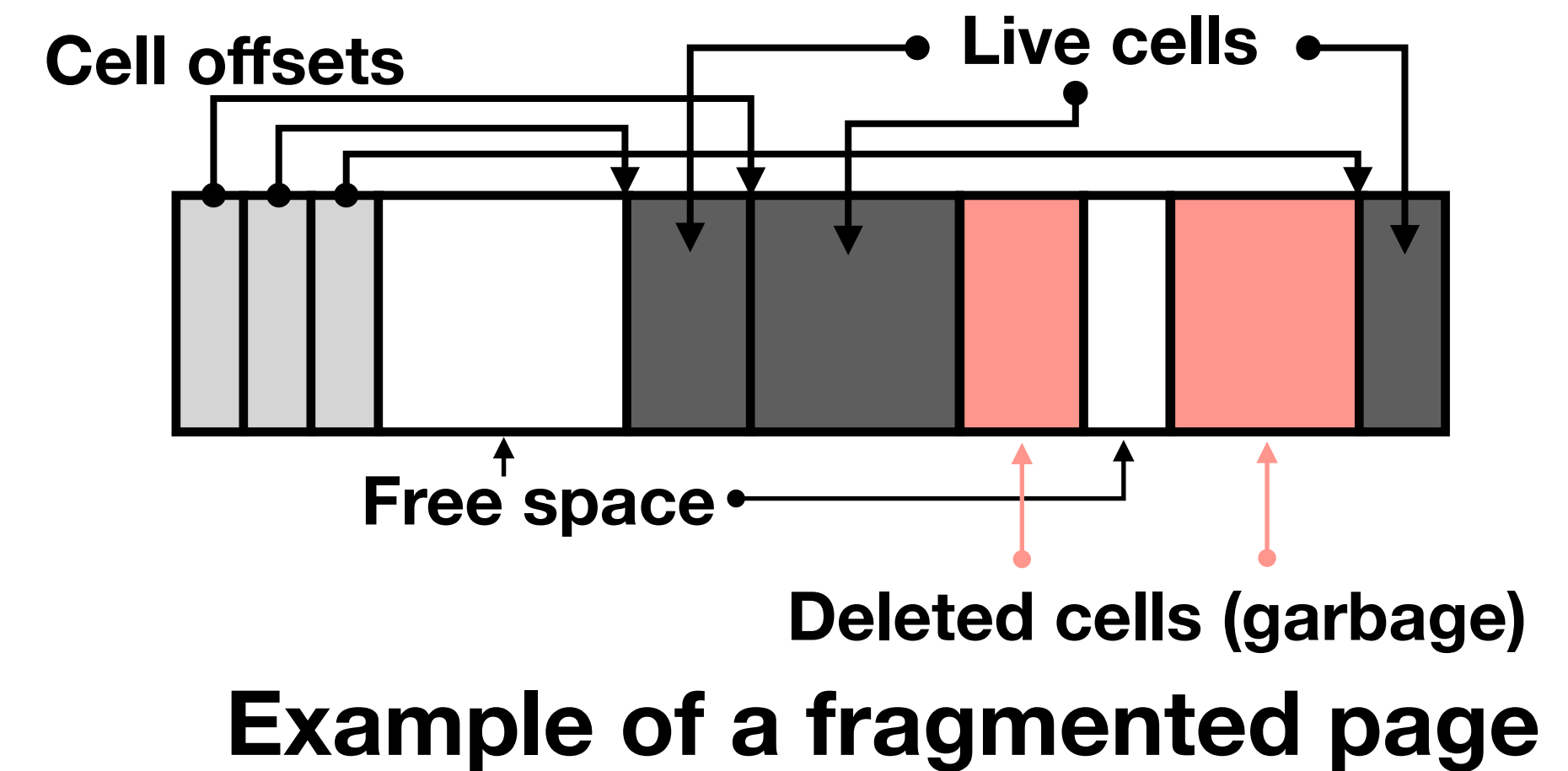
A **node merge** occurs when the occupancy of neighboring nodes falls under a threshold (***underflow***).

***Note:** Some B+tree implementations postpone splits and merges through **rebalancing**.*

B+tree

Fragmentation and vaccum

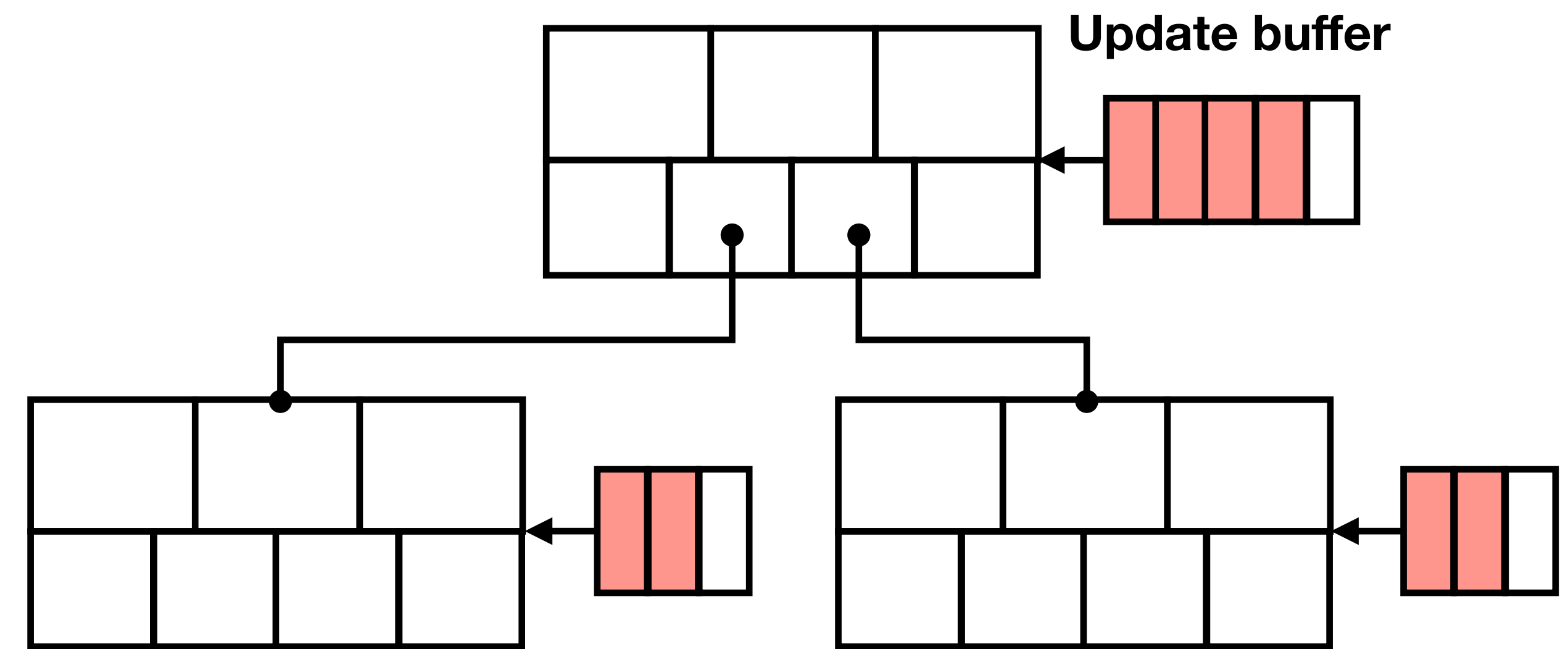
- Puts and deletes can cause fragmentation
 - Deletes only remove offsets/links, leaving the cell with non-addressable data
 - Insertion of key-value pairs with variable size may lead to empty free spaces
- *Vaccum* (or *compaction*) ensures page reclamation and rewrites cells in their logical order
 - High overhead!



B+tree

Implementation

- Copy-on-write B+trees (LMDB)
- **Lazy B+trees (WiredTiger)**
- Lazy adaptive tree
- FD-trees
- Bw-trees
- And many more



High-level design of WiredTiger

LSM tree

Organization

- ◎ **Log-structured merge trees** are indexing structures optimized for writes
 - Instead of *in-place updates*, data is **first written to memory** and then **asynchronously moved to persistent storage** into a single, larger write
 - *No random writes; all writes are sequential!*
- ◎ Contains two main components
 - An **in-memory memtable** absorbs all client writes
 - **On-disk levels** hierarchically store sorted key-value pairs in immutable files
- ◎ **High read and write amplification** (depending on the tuning knobs)

LSM tree

Organization

Skiplist for good read and write performance

Memory

Immutable
Memtable

memtable full



Memtable

Disk

L₀

SST

SST

Immutable files with
sorted key-value pairs

L₁

SST

SST

SST

L₂

SST

SST

SST

SST

...

...

L_N

SST

SST

SST

SST

SST

SST

SST

SST

*Each level is 10x
larger than the
previous one*

Larger levels, colder files

LSM tree

Read workflow

→ Foreground thread
--> Background thread

Memory

Immutable
Memtable

memtable full

Memtable

Get (40)

Block cache

Disk

L₀

SST

SST

L₁

SST

SST

SST

L₂

SST

SST

SST

SST

L_N

SST

SST

SST

SST

SST

SST

SST

SST

Each level is 10x
larger than the
previous one

Older key-value pairs may require traversing all levels

1. Bloom filters (reduce SST searches)

2. Index blocks (quick lookup within SSTs)

LSM tree

Read workflow summary

- ◎ **Foreground threads** handle the read workflow
- ◎ Reads are first submitted to the block cache
- ◎ If not found, are submitted to the memtable (and immutable memtable)
- ◎ Continue searching deeper levels if needed (L_0 , L_1 , ...)
 - Block cache is updated on the way back
- ◎ *Bloom filters* and *index blocks* can minimize read amplification and latency

LSM tree

Write workflow

→ Foreground thread
--> Background thread

Put (40, v5)

Memory

Immutable Memtable

memtable full

Memtable

Disk

L₀

SST

SST

L₁

SST

SST

SST

L₂

SST

SST

SST

SST

L_N

SST

SST

SST

SST

SST

SST

SST

SST

WAL

Flush

Compaction (L₀ -> L₁)

Compaction (L₁ -> L₂)

High write amplification!

Each level is 10x larger than the previous one

LSM tree

Compaction schemes: leveling merge policy

● Most common compaction scheme

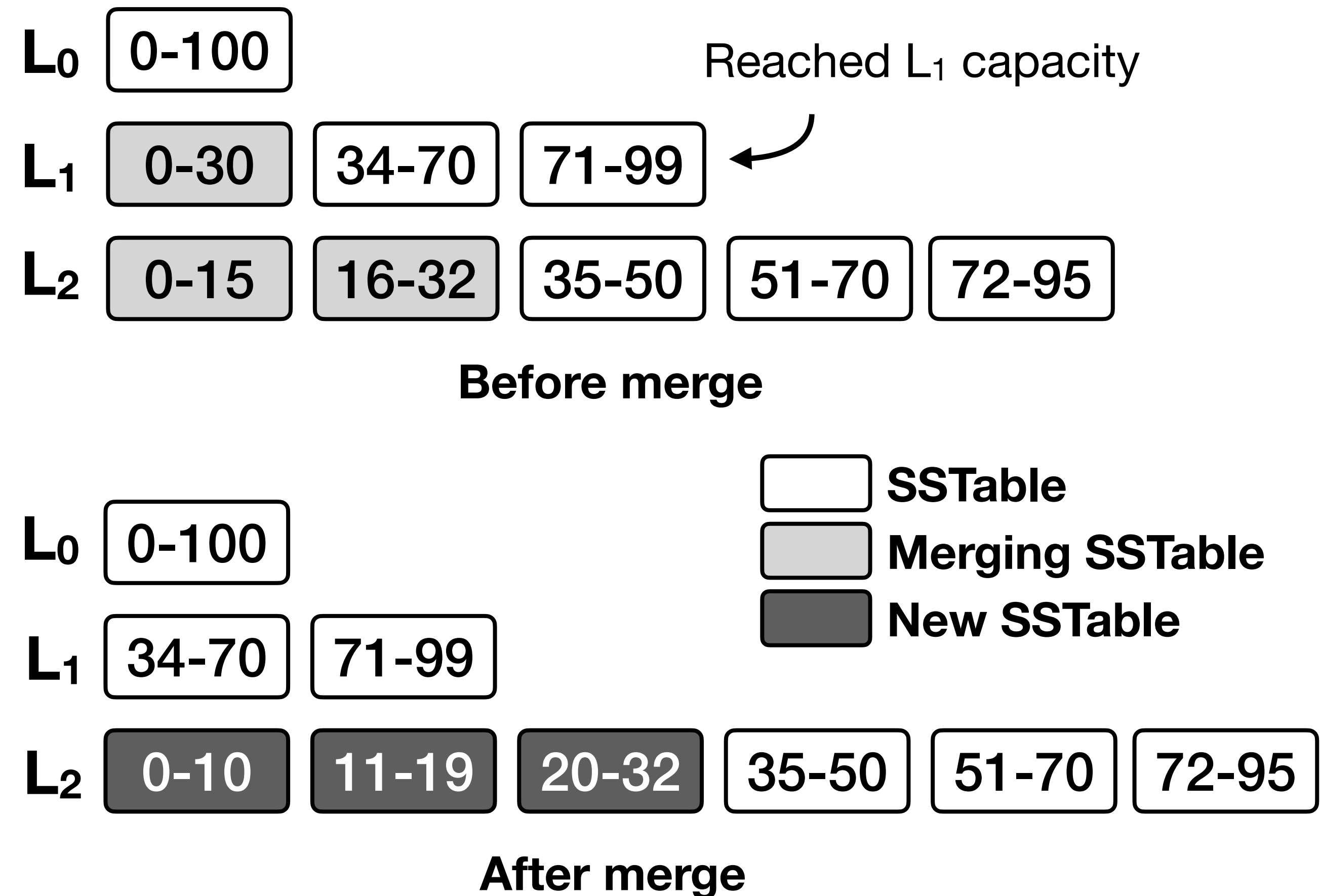
● Levels have a fixed-size ratio

● Older SSTs are gradually compacted into deeper levels

● Optimized for read-heavy workloads

- Only one SST per level is accessed on a read

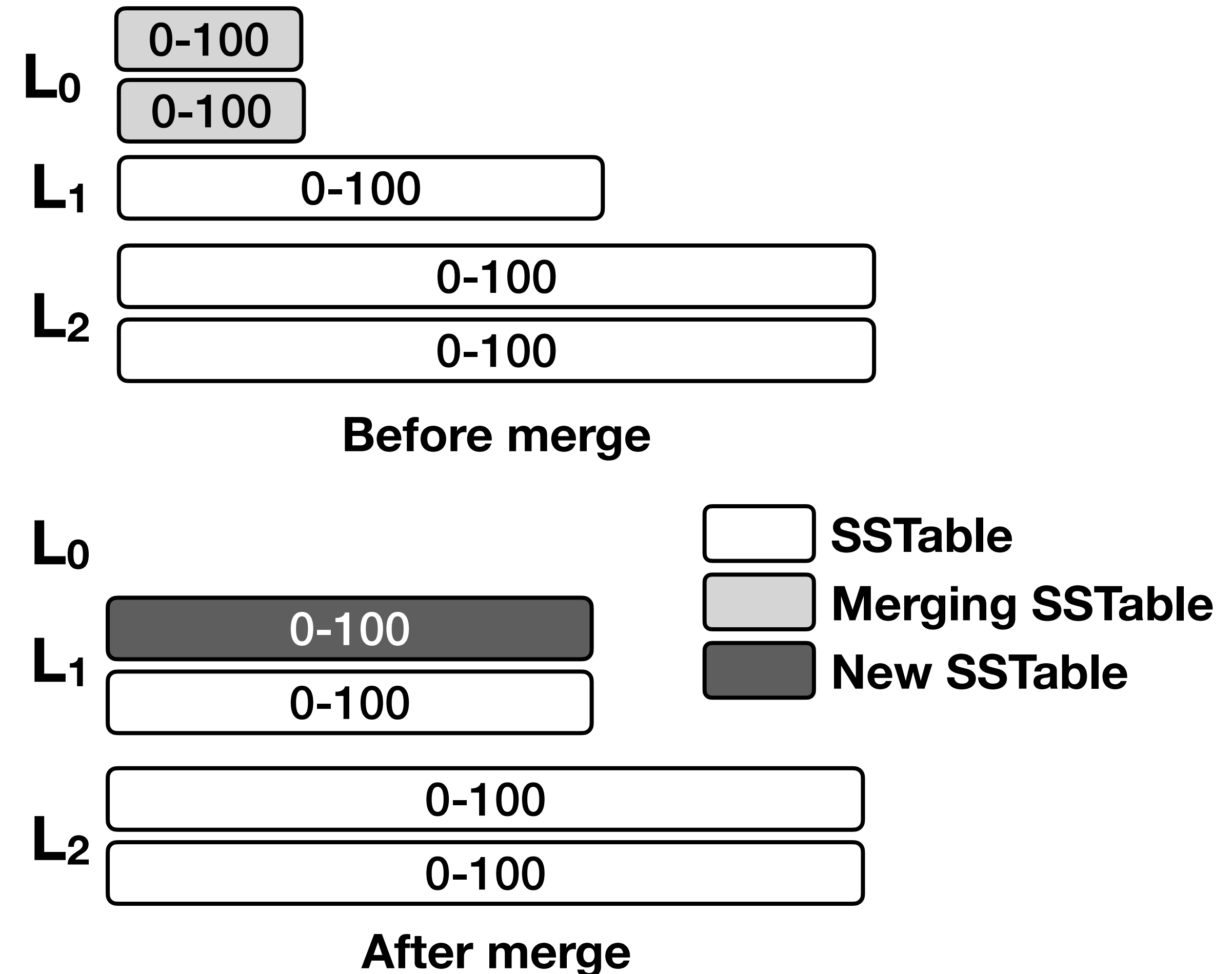
● High write amplification



LSM tree

Compaction schemes: tiering merge policy

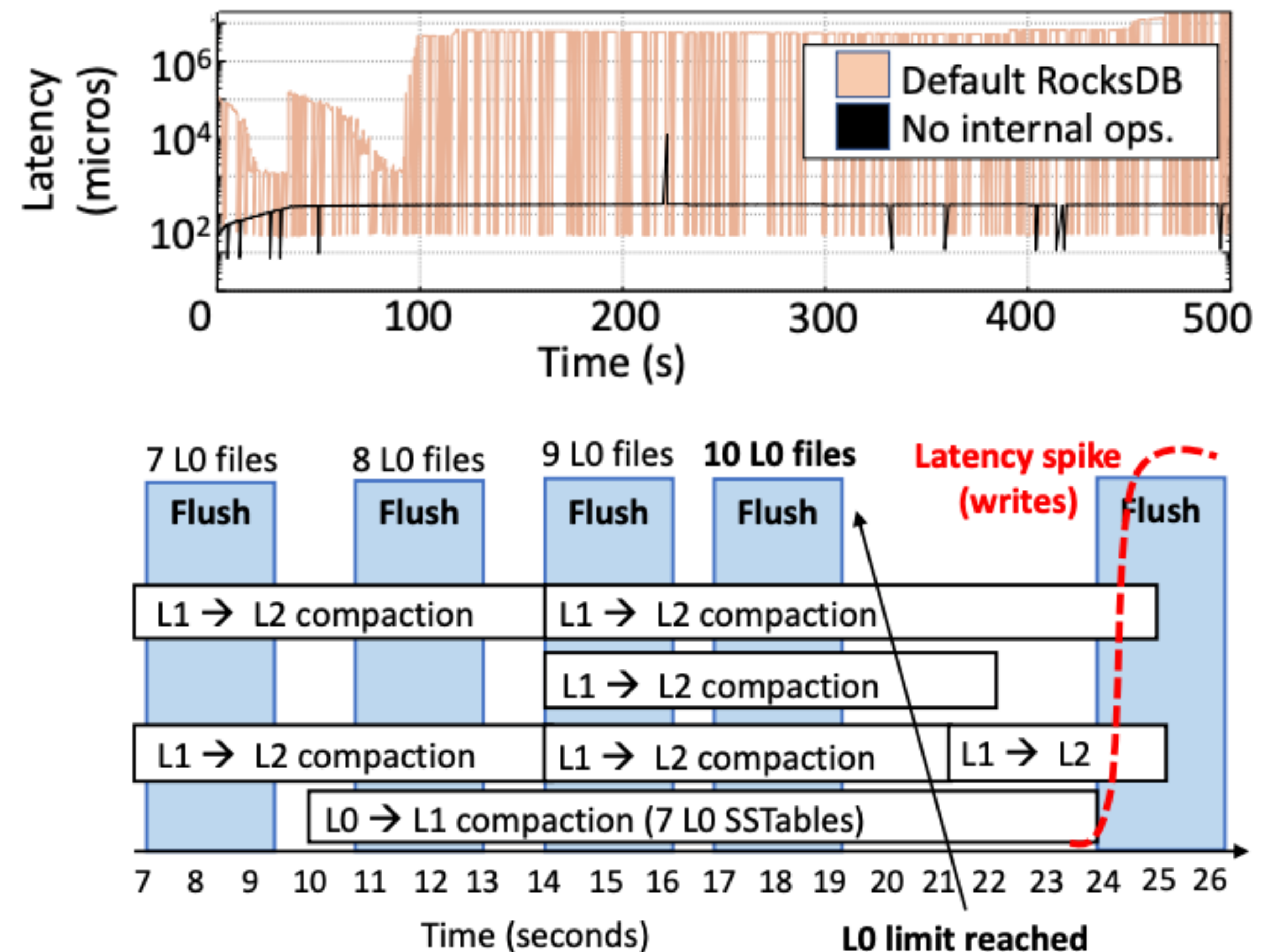
- New SSTs accumulate at each level rather than immediately merging
- When enough SSTs exist, they are merged into a larger SST
- SSTs have overlapping and duplicated key-value pairs
- Optimized for write-intensive workloads
 - Low write amplification
- High read amplification



LSM tree

Write stalls

- Some KVS support concurrent compactions
- Under prolonged write-intensive periods, concurrent operations can cause stalls and latency spikes
 - Slow flushes due to limited disk bandwidth
 - Slow low level compactions, temporarily halting flushes
- Solution: dynamic I/O scheduling!



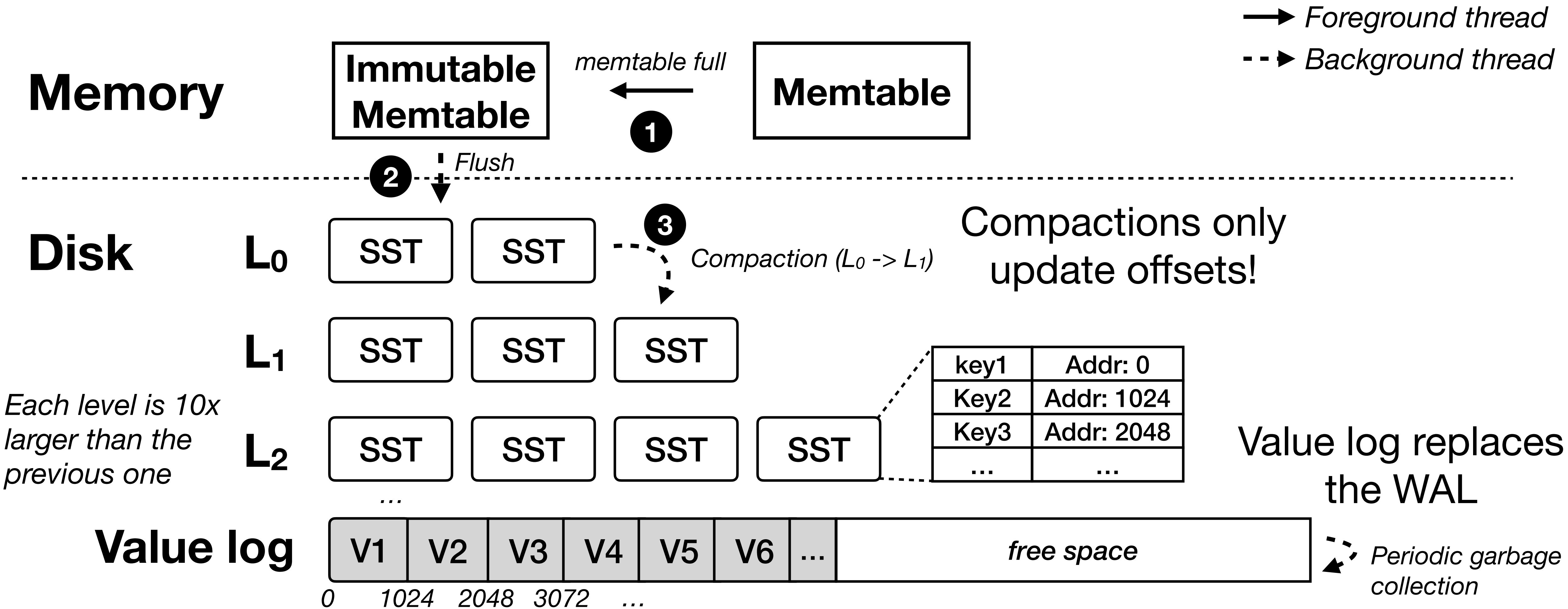
LSM tree

Write workflow summary

- Writes are absorbed in memory by the **memtable**
 - Turn multiple small-sized writes into a single larger one (**no random writes!**)
- When the memtable fills, it becomes immutable
 - The immutable memtable **flushes** its data to disk (L_0)
 - Unsorted key-value pairs are written directly as an SST
- When levels reach their capacity, a **compaction** is triggered
 - Merge-sort small SSTs into larger ones
 - Remove deleted data items
 - Become larger and take longer as we move down the LSM tree
- Stalls and latency spikes during prolonged write-intensive periods
- **Write amplification:** the same data is rewritten multiple times

LSM tree

Key-value separation (WiscKey)



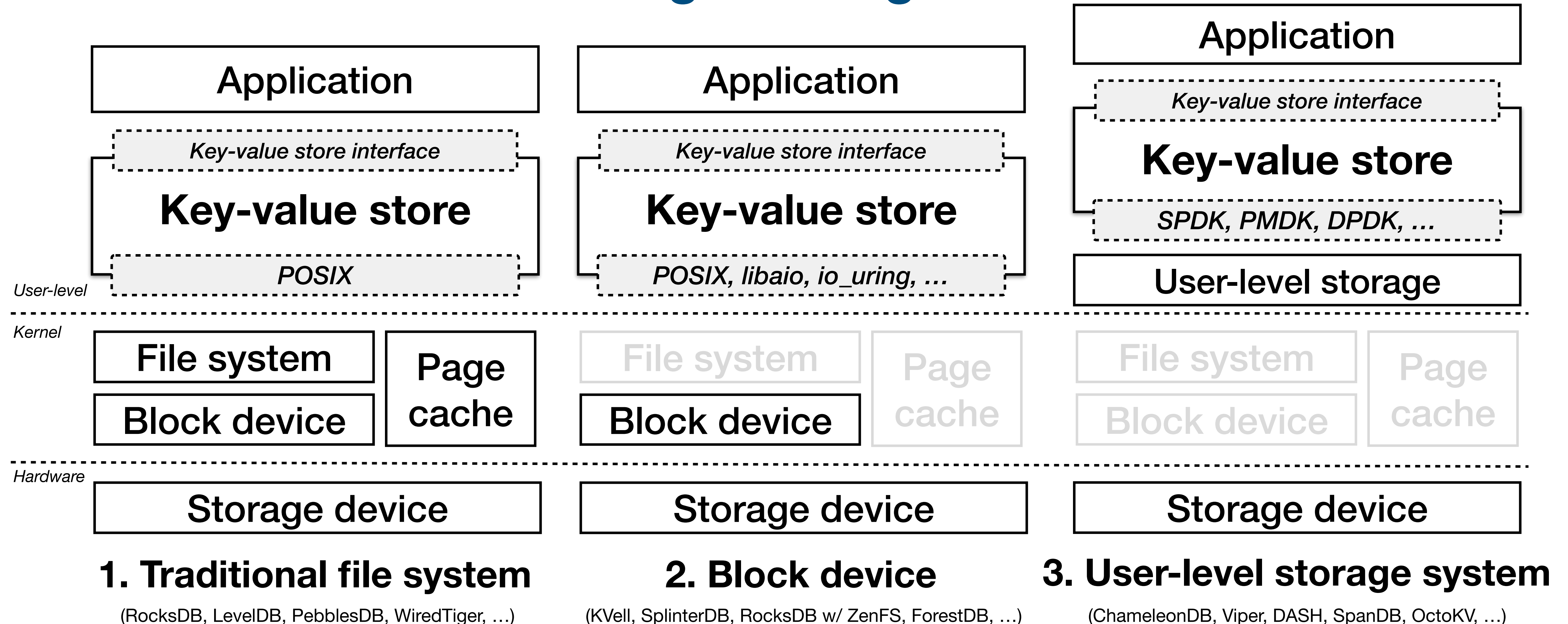
LSM tree

Storage management

- ◎ The underlying storage stack influences different aspects of KVS
 - Key-value pairs may be stored with different abstractions
 - File, block, memory address, physical key-value entry
 - Varying I/O path and storage interface
- ◎ Ultimately, both KVS and storage stack must be (re)designed to best service the application workloads!
 - Concurrency, key-value pair size, device capacity, consistency guarantees, ...

LSM tree

Storage management



LSM tree

(Some) Optimizations

- Heterogeneous storage devices
 - Tiering (Keigo), caching (P2Cache), storage disaggregation (Dinomo)
 - Performance vs. capacity vs. cost
- I/O management
 - Compression (LevelDB), encryption (Speicher), scheduling (SILK, PAIO), ...
- New data structures
 - Minimize read and write amplification (WiscKey, Monkey, ...)
 - Learned indexes (Bourbon)
- And many more

References

- Alex Petrov. **Database Internals: A deep dive into how distributed data systems work.** O'Reilly, 2019. Chapters 2, 4, 6, and 7.
- C. Luo and M. Carey. **LSM-based storage techniques: a survey.** *The VLDB Journal*, 2020.
- RocksDB wiki (<https://github.com/facebook/rocksdb/wiki>)
- Mark Callaghan. **Smalldatum blogposts** (<https://smalldatum.blogspot.com/>)
- S. Sarkar *et al.* **Constructing and Analyzing the LSM Compaction Design Space.** *Proceedings of the VLDB Endowment*, 2021.
- R. Adão *et al.* **Keigo: Co-designing Log-Structured Merge Key-Value Stores with a Non-Volatile, Concurrency-aware Storage Hierarchy.** *Proceedings of the VLDB Endowment*, 2025.